INSTITUTE OF SPACE TECHNOLOGY
KICSIT, Kahuta Campus

Course Title: Mobile Application Development

Assignment No. 2 - Theory Report

AI-Based Mini Application Development: Integrating ML/DL Models into Android with TensorFlow Lite

Project: Sentiment Analyzer (Kotlin + TensorFlow Lite)

Submitted By

Fatima Tehreem (242201073)

BSCS-5B

Course Instructor

Sir Uzair Hassan

Date

31 October 2026

Table of Contents

1. Introduction 2

2. Objectives 2

3. A1 - ML/DL in Android 2

3.1 Meaning . . . 2

3.2 Importance . . . 2

3.3 Benefits . . . 2

3.4 Challenges . . . 2

4. A2 - TensorFlow Lite 3

4.1 Definition . . . 3

4.2 Comparison with Standard TensorFlow . . . 3

4.3 Benefits . . . 3

4.4 Suitability for Android . . . 3

5. A3 - TFLite Integration in Android 3

6. The Original GitHub Project 4

6.1 Cloning . . . 4

6.2 Project Structure . . . 4

6.3 Code Analysis . . . 5

6.4 Dependencies . . . 6

6.5 Execution in Android Studio . . . 6

7. The Sentiment Analyzer App 7

7.1 Design . . . 7

7.2 Project Structure and Model . . . 8

7.3 Model File and Development Environment . . . 8

7.4 Build Configuration and Dependency . . . 9

7.5 Workflow and Code . . . 10

8. Testing and Results 12

9. Conclusion 13

10. References 13

1. Introduction

Machine learning (ML) and deep learning (DL) allow software to make predictions from data instead of following only fixed rules. This report documents Assignment No. 2 for Mobile Application Development. It explains how ML/DL models can be integrated into Android apps with TensorFlow Lite, analyses the instructor-provided GitHub project *NumberGuessingAppTflite*, and describes the original mini application built for the assignment: a **Sentiment Analyzer** written in Kotlin. The app lets a user type a sentence, and an on-device TensorFlow Lite model labels it as positive or negative and reports a confidence value.

2. Objectives

- To explain the meaning, importance, benefits and challenges of ML/DL in Android (A1).
- To define TensorFlow Lite, compare it with standard TensorFlow, and explain its suitability for Android (A2).
- To describe the steps of integrating a TFLite model into an Android app (A3).
- To study the original GitHub project: cloning, structure, code, dependencies and execution.
- To describe the design, model, workflow and testing of the Sentiment Analyzer app.

3. A1 - ML/DL in Android

3.1 Meaning

An ML model is a function learned from data; a DL model is an ML model built from multi-layer neural networks. In Android, a trained model can either run on a remote server that the app calls, or be packaged inside the app and run directly on the phone (*on-device inference*). Typical mobile uses include image classification, text classification, speech recognition and recommendations.

3.2 Importance

ML/DL lets mobile apps offer intelligent features, such as understanding text, images or speech, that would be difficult to write as fixed rules. Because phones hold personal data and are used in many situations without a reliable connection, running models on the device is an important way to deliver these features.

3.3 Benefits

- **Low latency:** no network round trip, so results appear quickly.
- **Offline use:** the app works without internet.
- **Privacy:** user data stays on the phone instead of being uploaded.
- **Smart features:** classification and prediction can be added directly to the user interface.

3.4 Challenges

- **Limited resources:** phones have limited CPU, memory and battery, so models must be small and efficient.
- **App size:** the model file and runtime library increase the APK size.
- **Accuracy trade-offs:** compact models can be less accurate than large server models; the confidence values observed in this project (Section 8) vary between inputs.

- **Updates:** a bundled model changes only when a new app version is released.

4. A2 - TensorFlow Lite

4.1 Definition

TensorFlow Lite (TFLite) is Google's lightweight runtime for running trained ML models on mobile and embedded devices. A model is usually trained with TensorFlow and then converted to the compact `.tflite` format for use in an app.

4.2 Comparison with Standard TensorFlow

Aspect	TensorFlow	TensorFlow Lite
Main purpose	Training and running models of any size	Running trained models on mobile/edge devices
Typical environment	Servers, workstations, cloud	Android, iOS, embedded devices
Model format	SavedModel / Keras formats	Compact .tflite file
Footprint	Larger	Small, optimised for low memory
Training	Supported	Not intended for training; inference only

4.3 Benefits

- **Speed:** optimised for fast on-device inference, avoiding network delay.
- **Size:** the converted model and runtime are compact, which suits app packages.
- **Efficiency:** lower memory and power use than running a full framework on a phone.

4.4 Suitability for Android

TFLite provides Android libraries and a model-in-assets workflow, so a model file can be bundled with the app and loaded from Kotlin or Java. Both projects in this report use it: the original repository through the low-level `Interpreter` API and the Sentiment Analyzer through the `NLClassifier` class of the Task Library.

5. A3 - TFLite Integration in Android

The integration steps are listed below, together with what each step looked like in this project.

1. **Convert the model to .tflite.** A trained TensorFlow/Keras model is converted to the `.tflite` format using the TensorFlow Lite converter. In this project the model files are `linear.tflite` (original) and `sentiment_model.tflite` (my app).
2. **Add the model to assets.** Create an `assets` folder under `src/main` and place the model file inside it (Figures 4, 12, 13).
3. **Add dependencies.** Add the TensorFlow Lite library to the module `build.gradle` and sync Gradle. My app uses `org.tensorflow:tensorflow-lite-task-text:0.4.4` (Figure 19).
4. **Load the model.** Original: `loadModelFile()` maps the model into a `MappedByteBuffer` for the `Interpreter` (Figure 7). My app: `NLClassifier.createFromFile(this, "sentiment_model.tflite")` (Figure 21).

5. **Prepare the input.** Original: the typed number is parsed into a `float[1]` array (Figure 6). My app: the typed text string is passed directly to the classifier.
6. **Perform inference.** Original: `interpreter.run(input, output)`. My app: `model.classify(text)` (Figure 22).
7. **Display the output.** Original: `output[0][0]` is shown in a `TextView`. My app: the highest-scoring label and its confidence percentage are shown with an emoji and a coloured card (Figures 23-24, 27-28).

6. The Original GitHub Project

The instructor-provided repository, *NumberGuessingAppTflite* by Uzair390-Del, is an Android app that feeds a single number into a TensorFlow Lite model and displays the numeric output.

6.1 Cloning

The project was cloned in Android Studio with the *Clone Repository* dialog (Figure 1): version control Git, the repository URL `https://github.com/Uzair390-Del/NumberGuessingAppTflite`, and a local directory named `NumberGuessingAppTflite` under Documents. The equivalent command-line form is:

```
git clone https://github.com/Uzair390-Del/NumberGuessingAppTflite.git
```

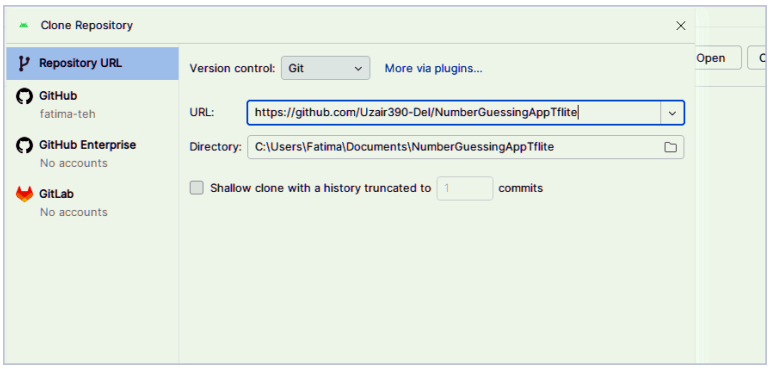


Figure 1: Clone Repository dialog in Android Studio.

6.2 Project Structure

Figure 2 shows the repository root: the folders `.idea`, `app` and `gradle`, and the files `.gitignore`, `build.gradle`, `gradle.properties`, `gradlew`, `gradlew.bat` and `settings.gradle`. All entries show the commit message "Initial commit". Inside the app module (Figure 3), `src/main` contains `assets`, `java/com/example/numberguessingtflite`, `res` and `AndroidManifest.xml`. The Android Studio view (Figure 4) shows the model `linear.tflite` in `assets`.

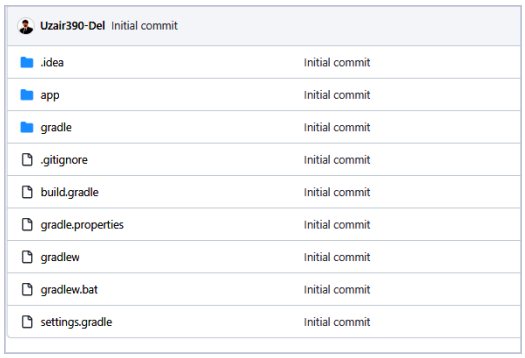


Figure 2: Root of the original GitHub repository.

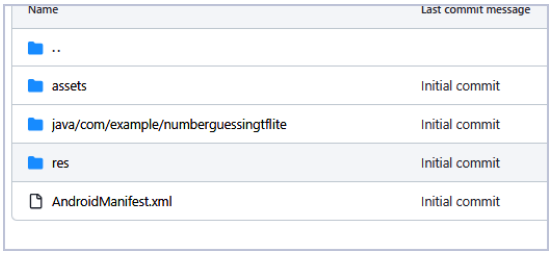


Figure 3: app/src/main contents in the original repository.

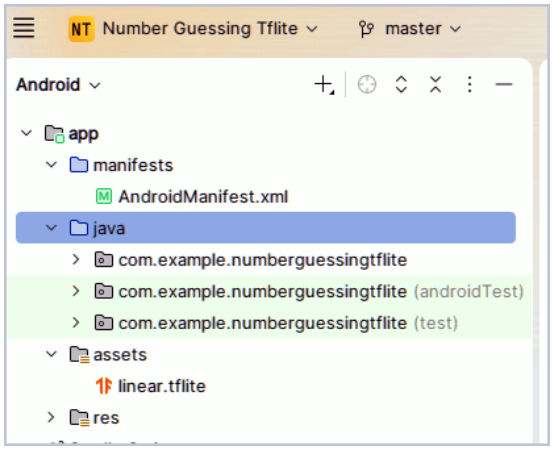
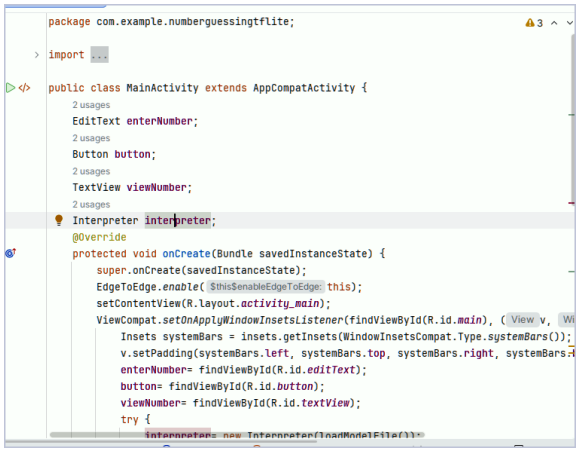


Figure 4: Original project in Android Studio, with linear.tflite in assets.

6.3 Code Analysis

The app is written in Java. Figure 5 shows the declarations in `MainActivity`: an `EditText` (`enterNumber`), a `Button`, a `TextView` (`viewNumber`) and a TensorFlow Lite `Interpreter`.



```
package com.example.numberguessingtflite;

import ...

public class MainActivity extends AppCompatActivity {
    2 usages
    EditText enterNumber;
    2 usages
    Button button;
    2 usages
    TextView viewNumber;
    2 usages
    Interpreter interpreter;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v,
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.
            enterNumber= findViewById(R.id.editText);
            button= findViewById(R.id.button);
            viewNumber= findViewById(R.id.textView);
            try {
```

Figure 5: Original MainActivity: view and Interpreter declarations.

In `onCreate` (Figure 6), the views are bound with `findViewById` and the interpreter is created with `new Interpreter(loadModelFile())`. In the button's click handler, the text is parsed into a `float` stored in a one-element input array, an output array of shape `[1][1]` is created, `interpreter.run(input, output)` performs inference, and `output[0][0]` is shown in the text view.

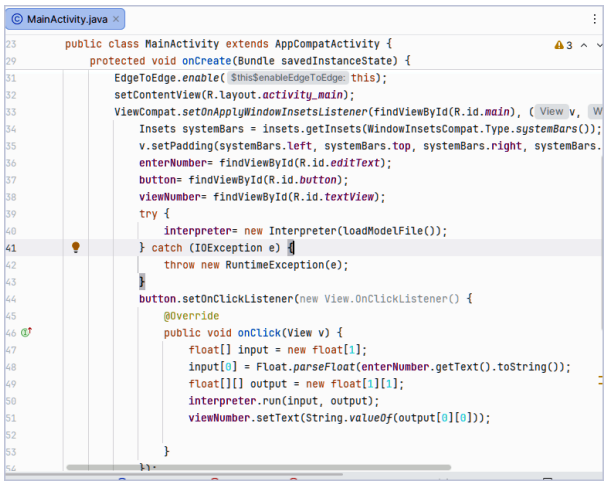


Figure 6: Original onCreate and click handler running the Interpreter.

The helper `loadModelFile()` (Figure 7) opens the model through the asset file descriptor, uses a `FileInputStream` and `FileChannel`, and maps the model region into memory as a read-only `MappedByteBuffer`, the form the Interpreter accepts.

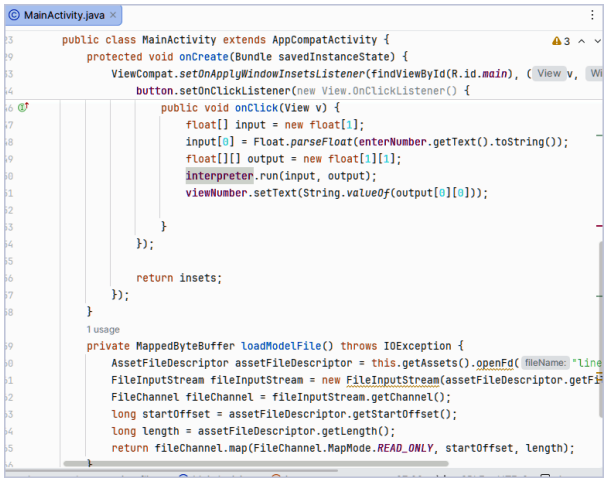


Figure 7: Original loadModelFile() method (memory-mapped model loading).

6.4 Dependencies

The original app's code imports and uses the TensorFlow Lite `Interpreter` class, so the TensorFlow Lite library must be available to the project. In contrast, my app adds the Task Library for text classification (Section 7.4).

6.5 Execution in Android Studio

The general procedure is: (1) open the cloned folder in Android Studio; (2) wait for Gradle sync to finish; (3) select an emulator or a connected Android phone; (4) click *Run*. Figure 8 shows the original app running on the test phone: the title is "Number Guessing Tflite", the input is **3**, and after tapping **SEND** the app displays **5.0001836**, which is the model output shown by the code in Figures 6 and 7.

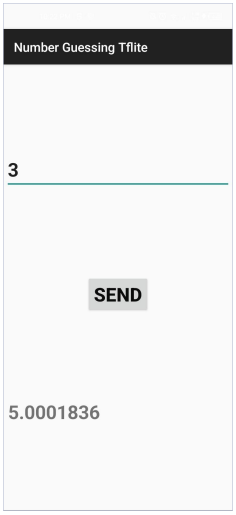


Figure 8: Original app running on a phone: input 3, output 5.0001836.

7. The Sentiment Analyzer App

7.1 Design

The app has one screen defined in `activity_main.xml` (a `ScrollView` with `LinearLayout`). Figure 9 shows part of the XML beside the design preview: the title "AI Sentiment Analyzer", the tagline "Your words, understood by AI", an "Enter Your Text" card with a text box and the "Analyze Sentiment" button, a "Prediction Result" card, and the footer "Powered by TensorFlow Lite". Figure 10 shows the input field settings: a 150dp-high multi-line `EditText` with the hint "Write a review or comment here..." and `maxLength="500"`, followed by the button with id `analyzeButton`. Drawable files such as `card_white.xml`, `input_background.xml` and `result_background.xml` provide the card and input styling.

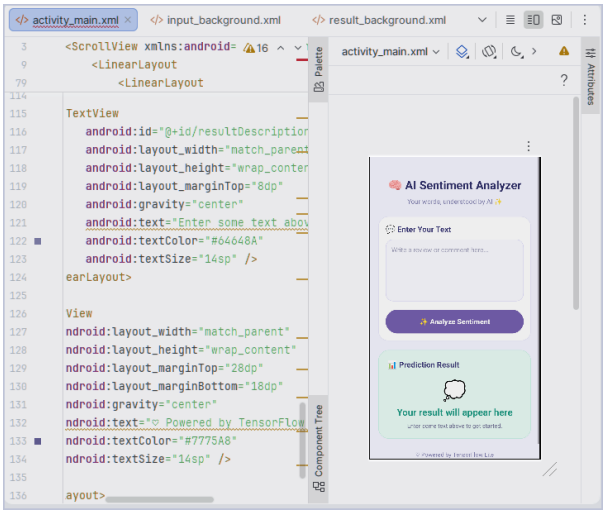


Figure 9: activity_main.xml and its layout preview in Android Studio.

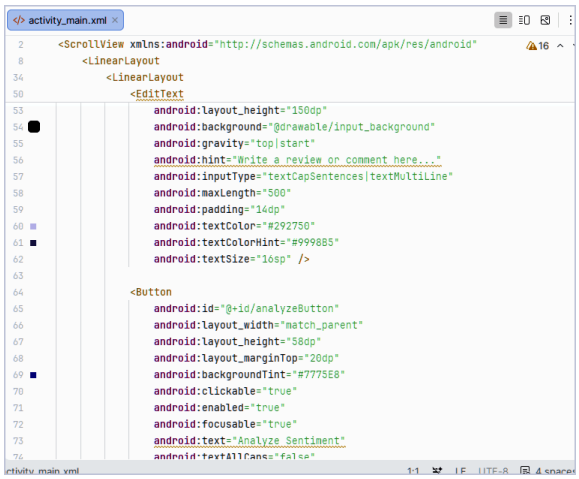


Figure 10: EditText (hint, 500-character limit) and Analyze button in activity_main.xml.

7.2 Project Structure and Model

Figure 11 shows the project `SentimentAnalyzer` with package `com.example.sentimentanalyzer` and `MainActivity.kt`. Figures 12 and 13 show the `assets` folder and the model `sentiment_model.tflite` inside it.

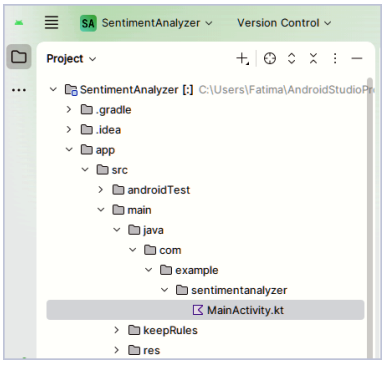


Figure 11: Sentiment Analyzer project and package structure.

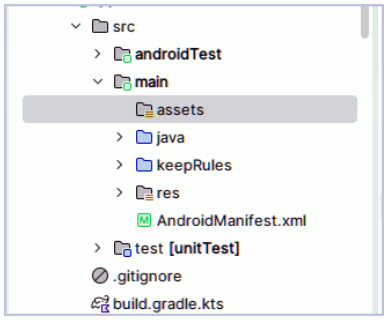


Figure 12: assets folder created in src/main.

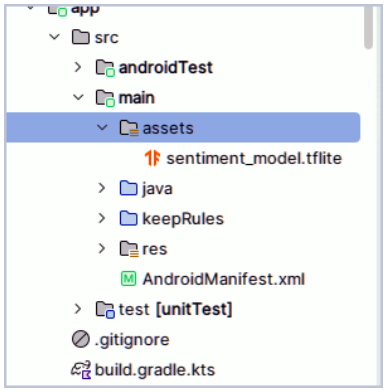


Figure 13: sentiment_model.tflite inside assets.

7.3 Model File and Development Environment

The file properties (Figure 14) show that `sentiment_model.tflite` is a TFLITE file of **751 KB (769,756 bytes)**, which supports the point that TFLite models are compact. The app was developed in **Android Studio Quail 4 | 2026.1.4** (Figure 15) and tested on a **TECNO CAMON 17** phone (model CG6j) running **Android 12** with a Helio G85 CPU and 6 GB RAM (Figure 16).

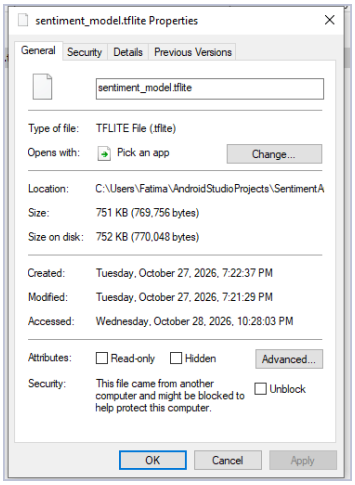


Figure 14: Properties of sentiment_model.tflite.



Figure 15: About Android Studio.

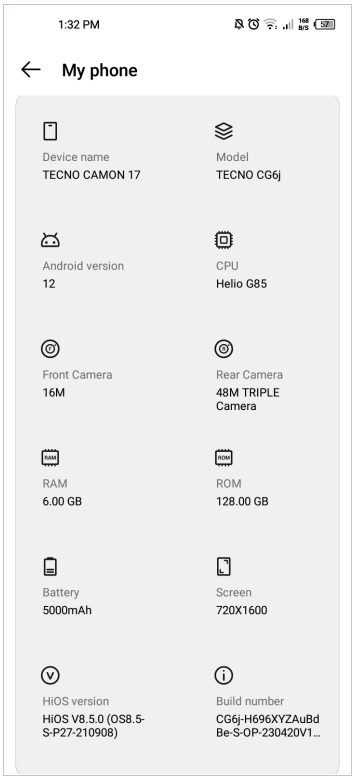


Figure 16: Test device information.

7.4 Build Configuration and Dependency

In `build.gradle.kts (:app)` (Figure 17) the namespace and application id are `com.example.sentimentanalyzer`, `compileSdk` and `targetSdk` are 37, `minSdk` is 24, the version is 1.0, and Java 11 compatibility is set. The version catalog `libs.versions.toml` (Figure 18) lists the Android Gradle Plugin `9.4.1` and the AndroidX/Material library versions. The TensorFlow Lite Task Library for text is added with `implementation("org.tensorflow:tensorflow-lite-task-text:0.4.4")` (Figure 19), so the developer does not manually handle the model buffer or input arrays as in the original repository.

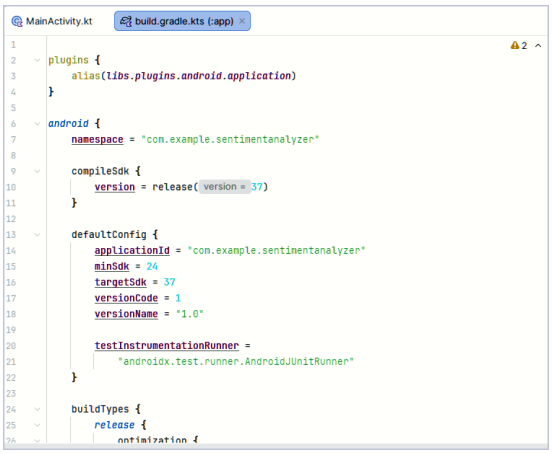


Figure 17: build.gradle.kts: namespace, SDK versions and application id.

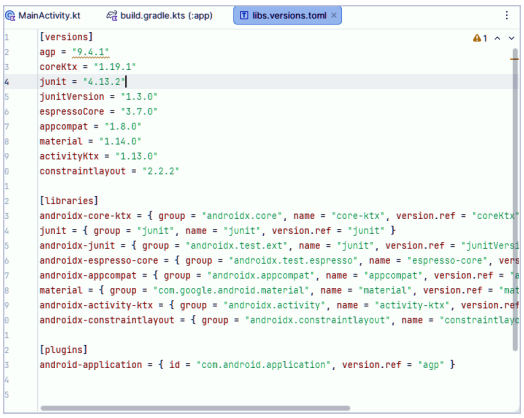


Figure 18: libs.versions.toml: plugin and library versions.

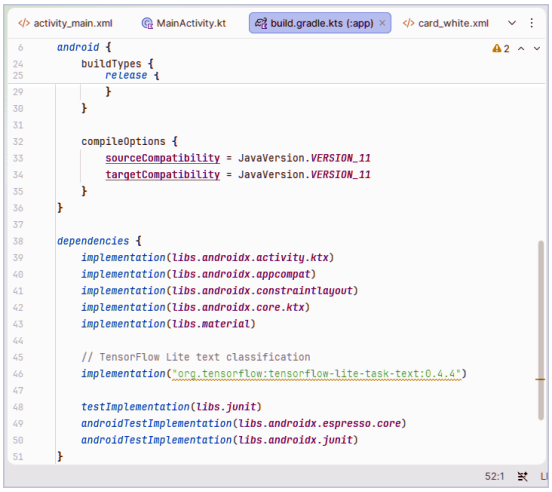


Figure 19: TensorFlow Lite Task Library (text) dependency in build.gradle.kts.

7.5 Workflow and Code

Workflow: the user types text and taps *Analyze Sentiment*; the classifier runs the model; the app selects the highest-scoring result; and the label and confidence are displayed. `MainActivity.kt` imports `NLClassifier` and declares `lateinit` views for the input, button and result fields (Figure 20).

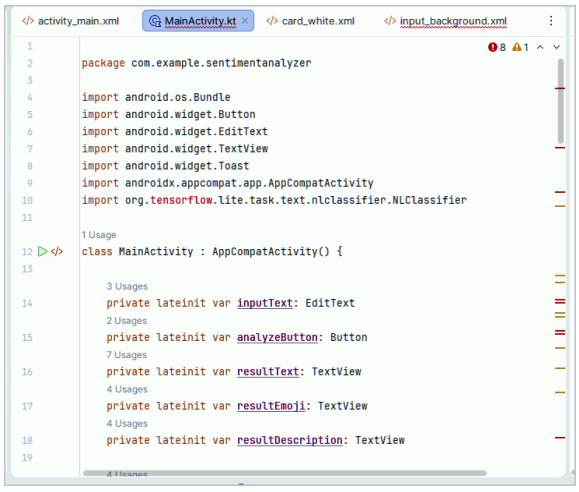


Figure 20: MainActivity.kt: imports and view declarations.

Model loading and input check. Inside `try`, the classifier is created with `NLClassifier.createFromFile(this, "sentiment_model.tflite")`; if loading fails, a `Toast` reports "Model could not load". In the click listener, the input is trimmed and, if empty,

the field shows the error "Please enter some text" and the method returns (Figure 21).

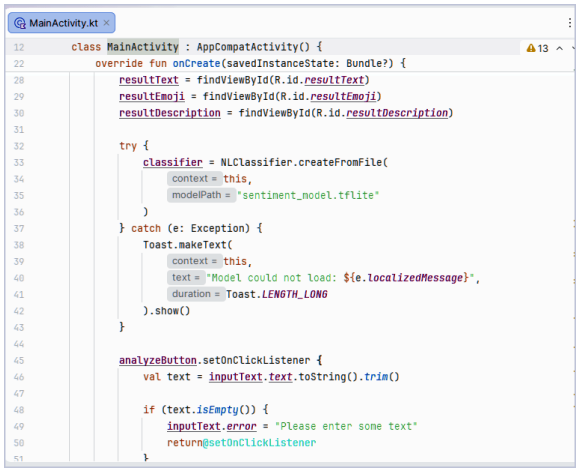


Figure 21: Model loading with error handling, and the empty-input check.

Prediction. If the classifier is null the app shows "Model not loaded". Otherwise it calls `model.classify(text)`. An empty result list shows "?", "No prediction" and "The model returned no results."; otherwise the best result is chosen with `maxByOrNull { it.score }` (if none, "Please try another sentence.") (Figure 22).

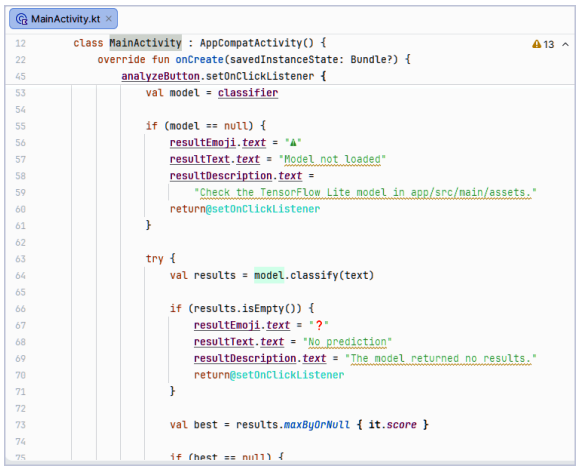


Figure 22: Null check, classify() call and handling of empty results.

Output. The label is normalised with `best.label.trim().lowercase()` and the confidence is `(best.score * 100).toInt()`. A `when` block maps labels containing "positive", or equal to "pos" or "1", to a smiling emoji, "Positive Sentiment" and a green text colour; labels containing "negative", or equal to "neg" or "0", to a sad emoji, "Negative Sentiment" and a red colour; any other label to "Model Result" with a neutral colour (Figures 23-24). The description line shows the model label and the confidence percentage.



```
if (best == null) {
    resultEmoji.text = "?"
    resultText.text = "No prediction"
    resultDescription.text = "Please try another sentence."
    return@setOnClickListener
}

val label = best.label.trim().lowercase()
val confidence = (best.score * 100).toInt()

when {
    label.contains( other = "positive") ||
            label == "pos" ||
            label == "1" -> {
        resultEmoji.text = "😊"
        resultText.text = "Positive Sentiment"
        resultText.setTextColor(
            Color.rgb( red = 22,  green = 143,  blue = 120)
        )
    }

    label.contains( other = "negative") ||
```

Figure 23: Mapping of positive labels (emoji, text, colour).



```
    label.contains( other = "negative") ||
            label == "neg" ||
            label == "0" -> {
        resultEmoji.text = "😞"
        resultText.text = "Negative Sentiment"
        resultText.setTextColor(
            Color.rgb( red = 205,  green = 75,  blue = 100)
        )
    }

    else -> {
        resultEmoji.text = "🔎"
        resultText.text = "Model Result"
        resultText.setTextColor(
            Color.rgb( red = 80,  green = 75,  blue = 150)
        )
    }
}

resultDescription.text =
    "Model label: ${best.label}\nAI confidence: $confidence%"
```

Figure 24: Negative and other labels, and the description text.

Error handling and cleanup. A `catch` block shows "Prediction failed" with the exception message, and `onDestroy()` closes the classifier and sets it to null to release resources (Figure 25).



```
                        Color.rgb( red = 80,  green = 75,  blue = 150)
                    )
                }
            }

            resultDescription.text =
                "Model label: ${best.label}\nAI confidence: $confidence%"

        } catch (e: Exception) {
            resultEmoji.text = "⚠"
            resultText.text = "Prediction failed"
            resultDescription.text =
                e.localizedMessage ?: "Please check the model file."
        }
    }
}

override fun onDestroy() {
    classifier?.close()
    classifier = null
    super.onDestroy()
}
}
```

sentimentanalyzer › MainActivity 12:7 LF UTF-8 4 spaces

Figure 25: Prediction error handling and onDestroy() cleanup.

8. Testing and Results

The app was run on the TECNO CAMON 17 phone and tested manually. Figure 26 shows the default screen before any input ("Your result will appear here", matching the default texts in the layout XML). Figures 27-28 show two inputs.

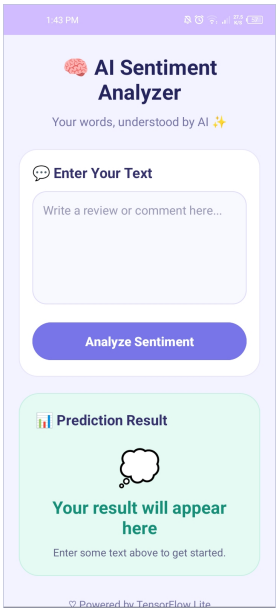


Figure 26: Default screen.

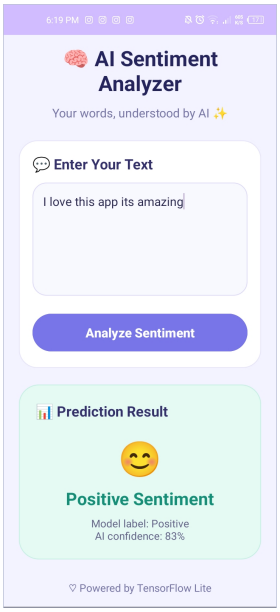


Figure 27: Positive input.

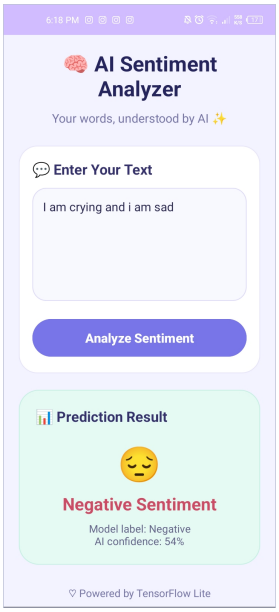


Figure 28: Negative input.

Test	Input	Observed output	Comment
1. Positive text	"I love this app its amazing"	Positive Sentiment; model label: Positive; confidence 83%	Correct classification.
2. Negative text	"I am crying and i am sad"	Negative Sentiment; model label: Negative; confidence 54%	Correct, with lower confidence.

Discussion. The positive and negative sentences were classified correctly, with confidences of 83% and 54%. The lower confidence for the negative sentence shows that confidence varies between inputs. Two inputs are a small sample and do not measure overall accuracy.

9. Conclusion

This assignment studied how ML/DL models can be embedded in Android apps with TensorFlow Lite. On-device inference offers low latency, offline use and privacy, at the cost of limited resources and model-size constraints. The original repository, which ran on the test phone (input 3 gave output 5.0001836), demonstrated the low-level `Interpreter` approach with a numeric model, while the Sentiment Analyzer used the higher-level `NLClassifier` to classify text in Kotlin. Testing on a TECNO CAMON 17 showed correct positive and negative predictions (83% and 54% confidence). Future work includes evaluating the model on a larger test set.

10. References

[1] Uzair390-Del. *NumberGuessingAppTflite* [GitHub repository]. https://github.com/Uzair390-Del/NumberGuessingAppTflite

[2] TensorFlow. *TensorFlow Lite guide*. https://www.tensorflow.org/lite

[3] TensorFlow. *TensorFlow Lite Task Library: NLClassifier*. https://www.tensorflow.org/lite/inference_with_metadata/task_library/nl_classifier